

Sonora — Documentação de Backend & Algoritmos

1. Visão Geral da Arquitetura

O Sonora é uma Progressive Web App (PWA) de streaming de música construída sobre a plataforma Base44, um backend-as-a-service. O frontend é uma Single Page Application em React 18 + Vite + Tailwind CSS; o backend (autenticação, base de dados, integrações e hosting) é fornecido pela plataforma. Não existe código de servidor escrito manualmente: toda a lógica de negócio vive no cliente — concentrada em `src/lib/PlayerContext.jsx` (motor de reprodução) e `src/lib/recommendations.js` (algoritmos de recomendação/shuffle) — e nas entidades/RLS configuradas via ficheiros JSON em `base44/entities/`. Esta arquitetura 'fat-client' é possível porque a Base44 expõe o SDK de entidades e integrações diretamente no browser, com Row-Level Security a impor o acesso aos dados no servidor.

2. Stack e Estrutura de Ficheiros

React 18 + React Router 6 (routing). Tailwind CSS + shadcn/ui (componentes em `src/components/ui/`) para a interface. lucide-react para ícones. recharts para gráficos (página Estatísticas). @hello-pangea/dnd para drag-and-drop de playlists. jspdf para geração de PDFs (Barra Secret). date-fns e moment para datas. Layout: `src/pages` (páginas), `src/components` (componentes reutilizáveis), `src/components/ui` (primitivas shadcn), `src/lib` (lógica: `PlayerContext`, `recommendations`, `AuthContext`, `zipBuilder`, `pdfBuilder`, `docsContent`, `sourceFiles`, `radioStations`), `base44/entities` (schemas + RLS), `public` (PWA: manifest, sw, ícone). O ponto de entrada é `src/main.jsx` e `src/App.jsx` (router).

3. Autenticação e Sessão

A autenticação é gerida pela plataforma via `AuthProvider` (`src/lib/AuthContext.jsx`). Suporta email+password, Google, GitHub e Meta. Fluxo de registo: `register` ! `OTP` ! `verifyOtp` ! `setToken`. `loginViaEmailPassword` faz redirect pós-login para o parâmetro `returnTo` (fallback '/'). O token persiste indefinidamente até logout explícito (`base44.auth.logout`). `ProtectedRoute` (`src/components/ProtectedRoute.jsx`) guarda as páginas autenticadas, redirecionando não-autenticados para /login. O utilizador atual obtém-se por `base44.auth.me()`; dados extra (`display_name`, `photo_url`, `bio`, `cover_photo_url`, `share_activity`) persistem-se via `base44.auth.updateMe(data)`. Não se podem criar/importar registos `User` — os utilizadores entram por convite (`base44.users.inviteUser(email, role)`).

4. Entidades e Modelo de Dados (resumo)

As entidades são schemas JSON em `base44/entities/`. Cada registo tem campos built-in: `id`, `created_date`, `updated_date`, `created_by_id` (dono), `created_by` (email do criador). Catálogo (público, sem RLS): `Song`, `Album`, `Artist`, `Podcast`, `PodcastEpisode`. Pessoais (RLS por dono): `Favorite`, `ListeningHistory`. Sociais: `Friendship` (convites), `FriendActivity` (atividade de escuta pública), `Profile` (perfil público). `Playlist` (RLS: pública ou dono). `User` é built-in. Detalhe completo no PDF de Base de Dados.

5. Row-Level Security (RLS)

Cada entidade tem um bloco rls que define read/create/update/delete. Regras usam template variables `{{user.id}}`, `{{user.email}}`, `{{user.data.<field>}}` e operadores MongoDB (`$or`, `$in`, `$gte...`). Padrões: dados pessoais (`Favorite`, `ListeningHistory`, `FriendActivity`, `Profile`) ! `read/update/delete` `user.id`; `create` aberto. `Playlist` ! `read` se `is_public` true OU dono; `writes` só d... veem e gerem (`created_by_id` OU `addressee_email` == `user.email` OU `addressee_id` == `user.id`). `FriendActivity` e `Profile` ! `read` aberto a todos os autenticados (para os amigos verem), `writes` `anchor` mais fiável (sempre presente).

6. PlayerContext — O Motor de Reprodução

`src/lib/PlayerContext.jsx` é o coração da app. Cria um React Context que envolve toda a aplicação (via `PlayerProvider` no Layout). Mantém estado global: `queue` (array de músicas), `currentIndex`, `isPlaying`, `currentTime`, `duration`, `volume` (default 0.7), `shuffle` (bool), `repeat` ('off'|'all'|'one'), `upNext` (fila prioritária), `djMode` (bool), `showNowPlaying` (bool), `isLoadingRadio`. Um único elemento `HTMLAudio` (`new Audio()`) é criado num `useEffect` na montagem e os seus eventos (`timeupdate`, `loadedmetadata`, `ended`, `play`, `pause`) sincronizam o estado React. `currentSong` = `queue[currentIndex]` (ou null se fora de range). Vários refs (`audioRef`, `endedRef`, `djModeRef`, `volumeRef`, `fadingRef`, `activityIdRef`) mantêm valores mutáveis entre renders sem re-render.

7. Algoritmo: Carregar e Reproduzir (playSong / playQueue)

playSong(song, newQueue, index=0): substitui a queue por newQueue (ou [song]), define currentIndex, limpa upNext, marca isPlaying=true e despacha o evento window 'music:play-song' (para fechar pop-ups, etc.). playQueue(songs, startIndex=0): igual mas exige songs não-vazio. Ambos não tocam diretamente — o useEffect que observa currentSong?.id é que carrega o audio.src e chama audio.play(). Isto centraliza a lógica de carga num só sítio: quando o id da música atual muda, o efeito define audio.src = currentSong.audio_url, chama audio.play() (com catch para bloqueios de autoplay), escreve o registo ListeningHistory e (se share_activity != false) cria o FriendActivity.

8. Algoritmo: Histórico e Atividade Social por Reprodução

No efeito de mudança de música (dependência [currentSong?.id]): (1) cria um registo ListeningHistory { song_id, song_title, artist_name, cover_url, played_at } — sempre, para o histórico pessoal. (2) se user && user.share_activity != false, cria um FriendActivity { song_id, song_title, artist_name, cover_url, is_playing: true } e guarda o id retornado em activityIdRef.current. Depois, busca os 30 FriendActivity mais recentes do utilizador (filter created_by_id == user.id, sort -created_date) e, se houver mais de 20, apaga os excedentes com deleteMany({ id: { \$in: ids } }). Isto mantém máximo 20 registos de atividade por utilizador (janela deslizante). Nos eventos play/pause do audio, atualiza-se FriendActivity.is_playing (true/false) via update no activityIdRef. Assim, os amigos veem em tempo real (realtime) se o utilizador está a ouvir e o quê.

9. Algoritmo: Navegação na Fila (next / prev / handleEnded)

next(): se upNext tem itens, tira o primeiro ([nextUp, ...rest]), insere-o na queue na posição idx+1 (splice) e avança para idx+1; senão, se idx+1 < queue.length avança; senão, se repeat=='all' volta a 0; senão fica. prev(): se currentTime > 3s, reinicia a música (currentTime=0) em vez de retroceder (comportamento padrão de players); senão decrementa currentIndex. handleEnded (atribuído a endedRef, chamado no evento ended): se repeat=='one', reinicia a mesma música (currentTime=0 + play); senão se upNext tem itens, insere e avança (como next); senão se idx+1 < queue.length avança; senão se repeat=='all' volta a 0; senão chama extendWithRadio() (gera mais músicas) e mantém idx. A fila prioritária upNext permite 'Tocar a seguir' sem alterar a queue original — os itens são inseridos dinamicamente à frente.

10. Algoritmo: upNext (Adicionar à fila / Tocar a seguir)

addToQueue(song) faz setUpNext(u => [...u, song]) — adiciona ao FIM da fila prioritária (toca depois de todos os upNext atuais). playNext(song) faz setUpNext(u => [song, ...u]) — adiciona ao INÍCIO (toca imediatamente a seguir à música atual). O consumo de upNext acontece em next()/handleEnded: o primeiro elemento é removido, inserido na queue em idx+1 e currentIndex avança. Isto implementa a semântica 'Up Next' do Apple Music: uma fila temporária que não destrói a queue principal.

11. Algoritmo: Shuffle (toggleShuffle)

toggleShuffle alterna o estado shuffle. Ao ATIVAR (e queue.length > 1): separa a música atual (queue[currentIndex]) das restantes, baralha APENAS as restantes com Fisher-Yates in-place, e reconstrói a queue como [current, ...remaining], com currentIndex=0. Isto garante que a música a tocar não salta e que as próximas ficam aleatórias. Ao DESATIVAR: apenas muda a flag (não reordena — a queue mantém a ordem atual baralhada). Fisher-Yates: percorre de trás para frente, troca cada elemento com um índice aleatório <= i (Math.floor(Math.random()*(i+1))). É O(n) e produz uma permutação uniforme.

12. Algoritmo: Repeat (cycleRepeat)

cycleRepeat alterna off! all! one! off. 'one': no ended, reinicia a mesma música e avança. 'all': no fim da queue volta ao índice 0 (loop infinito da fila). 'off': no fim, se não há mais músicas, chama extendWithRadio para continuar com recomendações (a 'fila infinita'). O estado repeat é lido em handleEnded e next via closure (useCallback com dependências [repeat, upNext, queue.length]).

13. Algoritmo: Recomendações (getRecommendations) — Rádio Personalizada

src/lib/recommendations.js. Carrega todas as músicas uma vez (loadAllSongs: Song.list('-play_count', 500), com cache em songsCache para evitar refetchs na sessão). Para um seed (song/artist/genre), deriva seedArtist, seedGenre, seedAlbum: se seed_type='song', busca a música e usa o seu artist_id, genre, album_id; se 'artist', usa seed_id como artist e infere o género da primeira música do artista; se 'genre', usa só o género. Depois

pontua cada música (excluindo o seed e exclude_ids): +50 se mesmo artista, +25 se mesmo gênero (case-insensitive), +15 se mesmo álbum, +min(15, play_count/100) por popularidade (tecto de 15 para não dominar), +Math.random()*5 de aleatoriedade leve (mantém frescor entre execuções). Filtra score>0, ordena por score desc, corta em limit (default 50). Isto produz uma rádio que prioriza fortemente o artista seed, depois gênero, depois álbum, com um impulso de popularidade e ruído aleatório para variar. É heurístico (sem ML) mas eficaz e barato.

14. Algoritmo: Shuffle de Artista (getArtistShuffleQueue / playArtistShuffle)

getArtistShuffleQueue(artist_id): filtra as músicas do artista, copia o array e aplica Fisher-Yates completo (baralha tudo, sem fixar nenhuma). Retorna a lista aleatória. playArtistShuffle(artistId): chama getArtistShuffleQueue, e se houver resultados faz playQueue(queue, 0) e setShuffle(true). Isto é o 'Shuffle Artist' do Apple Music: toda a discografia do artista em ordem aleatória, com a flag shuffle ativa.

15. Algoritmo: shuffleArray (utilitário)

shuffleArray(arr): Fisher-Yates puro sobre uma cópia ([...arr]), retorna um novo array baralhado. Usado em AlbumDetail (botão shuffle do álbum: playQueue(shuffleArray(songs), 0)) e onde se precise de uma permutação aleatória sem efeitos colaterais no array original.

16. Algoritmo: startRadio (Criar Estação)

startRadio(song): chama getRecommendations({ seed_type:'song', seed_id: song.id, limit:50 }) e monta a estação como [song, ...recs] (a música seed primeiro, seguida das recomendações), depois playQueue(station, 0). Em caso de erro, faz fallback para playQueue([song], 0). É a 'rádio a partir de uma música' — usa o algoritmo de scoring para construir uma fila infinita de música relacionada.

17. Algoritmo: extendWithRadio (Fila Infinita)

extendWithRadio(): quando a queue acaba (e repeat != 'all'/'one'), chama getRecommendations com seed_type 'song', seed_id = currentSong.id, exclude_ids = todos os ids já na queue (para não repetir) e limit 25. Se houver resultados, faz setQueue(q => [...q, ...recs]) (anexa as recomendações) e setCurrentIndex(idx => idx+1) (avança para a primeira recomendação). Tem guarda isLoadingRadio para não disparar em paralelo. Isto cria a experiência de 'fila infinita': quando a tua fila acaba, o Sonora continua automaticamente com música relacionada, como uma estação de rádio baseada na última música.

18. Algoritmo: Modo DJ e Crossfade

toggleDjMode alterna djMode. fadeAudio(audio, to, duration): interpola linearmente o volume de fromVol para to ao longo de duration ms usando requestAnimationFrame (passo a cada frame: volume = fromVol + (to-fromVol)*t, onde t = min(1, elapsed/duration)). No evento 'play' do audio, se djMode ativo: audio.volume=0 e fadeAudio(audio, volumeRef.current, 2500) — fade-in de 2.5s no início de cada música. Um useEffect observa [currentTime, duration, djMode, isPlaying]: se djMode ativo e faltam < 4s para o fim e ainda não está a fazer fade, marca fadingRef e fadeAudio(audio, 0, max(800, remaining*1000)) — fade-out progressivo até ao fim. Ao desativar o DJ mode, restaura o volume e limpa fadingRef. Isto cria transições suaves (crossfade-like) entre músicas, estilo rádio/DJ, sem bibliotecas externas.

19. Algoritmo: Adicionar a Playlist (AddToPlaylistDialog)

Ao abrir, carrega as playlists do utilizador (Playlist.list('-updated_date', 100)). Para cada playlist mostra capa, título e contagem (song_ids.length) e um ícone Check se a música já lá está. add(playlist): lê song_ids; se o song.id já está incluído, mostra toast 'Já está na playlist' e fecha; senão chama Playlist.update(p.id, { song_ids: [...ids, song.id], track_count: ids.length+1 }) — anexa o id e atualiza a contagem. Em caso de erro, toast de erro. Isto mantém a denormalização (track_count) sincronizada com song_ids no próprio write, evitando queries de contagem.

20. Algoritmo: Criar Playlist (Sidebar)

createPlaylist(): Playlist.create({ title:'Nova playlist', song_ids:[], track_count:0, is_public:false }) e navigate(`/playlist/\${p.id}`). A nova playlist aparece imediatamente na sidebar porque o Layout subscrive alterações à entidade Playlist (realtime): no evento 'create', insere o novo registo no topo da lista local; no 'update', substitui; no 'delete', remove. Assim, criar/editar/apagar playlists reflete-se instantaneamente na sidebar sem reload.

21. Algoritmo: Tags de Playlist nos Álbuns (AlbumDetail)

Ao abrir um álbum, além das músicas, carrega-se as playlists do utilizador (Playlist.list). Constrói-se um mapa `song_id!` [títulos de playlists que contêm essa música]: para cada playlist, adiciona o título ao array do `song_id` (sem duplicados). Passa-se `playlistTags={map[s.id]}` ao `SongRow`. O `SongRow` renderiza até 2 pills (tags) rosa com o nome da playlist; se houver mais que 2, mostra um '+N' que, ao passar o rato por cima, abre um pop-up (group-hover/tag) com a lista completa. Isto permite ver, à frente da duração, em que playlists cada música do álbum já está.

22. Algoritmo: Funcionalidade de Amigos

Entidades: Friendship (convites/amizades), FriendActivity (atividade pública), Profile (perfil público). Convite: um utilizador cria Friendship { addressee_email, status:'pending' } e envia email via Core.SendEmail. O destinatário vê o convite em /amigos! Convites (filtrando Friendship onde addressee_email != ele próprio). Aceitar: update status='accepted' + addressee_id = o seu id. A página Amigos lista amigos aceites (a 'outra parte' de cada Friendship accepted), mostra a atividade mais recente de cada um (FriendActivity agrupado por created_by_id, mais recente primeiro) e abre um pop-up de perfil (FriendProfileModal) com playlists públicas + 5 músicas recentes do amigo. Toggle de privacidade (User.share_activity): se desligado, o PlayerContext NÃO cria FriendActivity e apaga os registos existentes (o utilizador fica 'offline' para os amigos).

23. Páginas e Routing

src/App.jsx define as rotas dentro de <ProtectedRoute> + <Layout>: / (Home/Ouvir agora), /search, /radio, /library/:section, /playlist/:id, /artist/:id, /album/:id, /podcasts, /estatisticas, /amigos, /secret-bar. Layout (src/components/Layout.jsx) renderiza Sidebar (desktop) + MobileTabBar (mobile) + PlayerBar + NowPlayingView, tudo dentro de PlayerProvider. As páginas de auth (login, register, forgot, reset) ficam fora do ProtectedRoute. A rota /secret-bar só é acessível pesquisando 'secret' na barra de pesquisa (entra um card 'Barra Secret').

24. PlayerBar e NowPlayingView

PlayerBar (src/components/PlayerBar.jsx) é a barra persistente: info da música, controlos play/skip/shuffle/repeat, slider de progresso (seek), volume, modo DJ, criar estação, favoritar. Clicar numa zona não-funcional abre NowPlayingView (ecrã completo com capa, letras, fila, histórico). Quando nenhuma música está a tocar (sem currentSong), o PlayerBar devolve null (não aparece). NowPlayingView é um modal controlado por showNowPlaying no PlayerContext. O favoritar (handleLove) cria/apaga um registo Favorite e reflete no estado local 'loved'.

25. Integrações Core usadas

Core.InvokeLLM — respostas LLM (com add_context_from_internet para web search, file_urls para visão, response_json_schema para saída estruturada). Core.UploadFile — upload de imagens (capas de playlist, foto/capa de perfil). Core.SendEmail — convites de amizade e reportes. Core.GenerateImage — capas/imagens temáticas. Core.GenerateVideo/GenerateSpeech/TranscribeAudio/ExtractDataFromUploadedFile disponíveis. Chamadas via base44.integrations.Core.<Endpoint>(data). Detalhe no PDF de Integrações.

26. PWA e Offline

index.html declara manifest.json, theme-color, meta apple-mobile-web-app e regista /sw.js. public/manifest.json define name 'Sonora', display standalone, ícone SVG (any maskable), start_url '/', cores. public/sw.js faz precache de /, /index.html, /manifest.json, /icon.svg e stale-while-revalidate para GETs (cache dinâmico). Isto torna a app instalável e offline-capable para os estáticos; os dados (entidades) continuam a precisar de rede.

27. Imagens e Fallbacks

O componente Image (src/components/ui/image.jsx) serve imagens da plataforma Wix/Base44 redimensionadas ao container e re-encodadas em WebP, com placeholder blur-up (uma miniatura de 20px desfocada sobre o tempo de carga). FALLBACK_IMAGE_URL é uma capa genérica (anel de onda rosa sobre fundo escuro) usada quando uma capa falta ou falha. Todos os MediaCards e SongRows usam este componente. Outros hosts e SVGs passam inalterados.

28. Realtime (Subscriptions)

`base44.entities.<Name>.subscribe(callback)` permite reagir a eventos `create/update/delete` em tempo real. O Layout usa-o para manter a sidebar de playlists sincronizada (ver secção 20). O evento tem `{ id, type, data }`. Isto evita polls e garante que múltiplos clientes/tabs veem alterações imediatamente.

29. Considerações e Limitações

Letras são estimadas (sem ficheiros LRC nativos — sincronização aproximada). Streams de rádio dependem de APIs externas (Radio Browser API) sujeitas a disponibilidade/CORS. As recomendações são heurísticas (sem ML externo) — eficazes para catálogos pequenos/médios. A sincronização de atividade social é client-side (não há backend custom; o servidor só impõe RLS). O User não pode ser criado/importado (apenas por convite). A persistência do token é gerida pela plataforma. O cache de músicas (`songsCache`) vive na sessão e é limpo por `clearSongsCache()` quando se adicionam músicas novas.

30. Biblioteca — Artistas e Álbuns do Último Ano

`loadRecentArtists/loadRecentAlbums (src/pages/Library.jsx)`: carregam `ListeningHistory.list('-played_at', 500)`, filtram `played_at >= 1` ano (`cutoff = hoje - 1 ano`), recolhem `song_ids` únicos, fazem `Song.filter({ id: { $in: songIds } })` e derivam `artist_ids/album_ids` únicos ordenados por primeira aparição no histórico (audição mais recente). Depois `Artist.filter / Album.filter({ id: { $in: ordered } })` e mapeiam pela ordem. Só aparecem artistas/álbuns efetivamente ouvidos no último ano, por ordem de recência.

31. Paginação (LibraryPagination)

Componente reutilizável (`src/components/LibraryPagination.jsx`): chevrons + 'X de Y', minimalista. Usado em Artistas e Álbuns (`PAGE_SIZE=24`) e em Podcasts (`PAGE_SIZE=24`). Favoritos mantém-se sem paginação (tudo corrido, até 500 registos). A página reinicia para 1 ao mudar de secção.

32. Aba Playlists

Nova secção `/library/playlists (sidebar Biblioteca! Playlists)` que mostra `todo (Playlist.list('-updated_date', 500))`, distinta de 'Recentes' (100 mais recentes). `todo` passa a apontar para `/library/playlists`.

33. SectionDivider

Componente reutilizável (`src/components/SectionDivider.jsx`): linha gradual (`from-transparent` via `border-to-transparent`) com ícone central (default `Music2`). Usado em `PlaylistDetail` (acima de 'Músicas sugeridas', `Music2`), em `Radio` (acima de 'A tua estação principal' e 'Estações por género', `Radio`), em `Home/Ouvir Agora` (entre secções: `Tocadas recentemente`, `Top músicas`, `Novos lançamentos`, `As tuas playlists`, `Estação para ti`) e em `InterviewsSection (Mic2)`.

34. AirPlay

`PlayerContext` expõe `showAirplayPicker()` que chama `audioRef.current.webkitShowPlaybackTargetPicker()` — a API `WebKit` que abre o seletor de `AirPlay/saídas de áudio` do sistema. O botão `Airplay (lucide Airplay)` está na `PlayerBar`, cluster direito, visível a partir de md. Se o browser não suportar a API, a chamada é no-op (guardada com `typeof check`).

35. Entrevistas (Podcasts)

Nova secção `InterviewsSection (src/components/InterviewsSection.jsx)` na página `Podcasts`. Pesquisa por nome de pessoa! `iTunes Search API (media=podcast, term '{name} interview')`. `Recent startups: Elon Musk, Sam Altman, Satya Nadella, Reid Hoffman, Brian Chesky, Patrick Collison` buscadas no `mount` e filtradas por `isRecent(releaseDate, 1 mês)` — só entrevistas com episódio nos últimos 30 dias. Cada card liga a `trackViewUrl (Apple Podcasts)` via 'Abrir'. O algoritmo de `matching/playback` será refinado pelo utilizador; a baseline usa `iTunes Search + filtro de recência`.

36. Código de Acesso (gate privado)

A app é privada — só acessível com o código 'SonoraSecret'. Campo `User.access_verified (boolean, default false)`. Gate: `AccessGate (src/components/AccessGate.jsx)` envolve as rotas protegidas (dentro de `ProtectedRoute`, em `src/App.jsx`); se `user logado e !access_verified!` redireciona.

(src/pages/AccessCode.jsx): input do código; se === 'SonoraSecret' !' auth.u
checkUserAuth !' redirect '/'; senão mostra erro (e botão Sair/logout). Regist
input 'Código de acesso' validado ANTES de base44.auth.register (deve ser 'SonoraSecret'); após verifyOtp +
setToken, auth.updateMe({ access_verified: true }) e só depois redirect. OAuth (login ou registo): a plataforma
cria o User automaticamente; novo user fica access_verified=false !' AccessC
!' só acede após inserir o código. Contas já existentes foram migradas para a
'mantem como esta' no login). Login de contas existentes (email/password ou OAuth) mantém-se sem código.
CÓDIGO DE ACESSO: 'SonoraSecret'.

37. Algoritmo: Músicas Essenciais (estrela, estilo Apple Music)

Algoritmo que marca as canções mais famosas de cada artista com uma estrela
como no Apple Music. Definição: para cada artista, até 5 músicas marcadas como 'essenciais'. Critério de
selecção: ordenar as músicas do artista por play_count desc e marcar as primeiras até 5 (top 5 por popularidade)
— só artistas com >= 5 músicas são elegíveis para ter faixas marcadas (artistas com poucas músicas não têm
'essenciais'). Implementação: getEssentialSongIds(artistId) em src/lib/recommendations.js — filtra
Song.filter({artist_id}, '-play_count', 100), recolhe os ids das primeiras 5 e retorna um Set; se o artista tiver < 5
músicas, retorna Set vazio (nenhuma marcada). Apresentação: o AlbumDetail e o ArtistDetail passam
essentialIds={getEssentialSongIds(artistId)} ao SongRow; o SongRow, se essentialIds.has(song.id), renderiza
um ícone Star (lucide-react) imediatamente antes do track_number (formato:
visual (não afeta reprodução nem ordenação) e indica as faixas mais populares/representativas do artista,
ajudando o utilizador a identificar os maiores êxitos num álbum ou discografia. O cálculo é feito por artista (cache
opcional em sessão) e baseia-se no play_count global (denormalizado em Song.play_count). Em álbuns, só as
faixas do artista desse álbum podem ficar marcadas; em ArtistDetail, todas as músicas do artista são elegíveis.
O limiar de 5 é fixo (configurável no futuro por artista, ex.: artistas muito grandes poderiam ter mais).

38. Algoritmo: Top Músicas (7 mais tocadas na última semana)

A secção 'Top músicas' do Home (Ouvir agora) apresenta as 7 músicas mais tocadas pelo utilizador na última
semana (não all-time). Implementação: carregar ListeningHistory.list('-played_at', 500) (RLS: só o histórico do
próprio utilizador), filtrar registos com played_at >= cutoff (agora - 7 dias), agrupar por song_id contando
reproduções, ordenar por contagem desc, tomar os primeiros 7 song_ids e fazer Song.filter({ id: { \$in: top7Ids } })
para obter os metadados (mantendo a ordem por contagem). Renderizar os 7 cards. Isto reflete os êxitos do
momento (últimos 7 dias) do próprio utilizador, dando frescor e personalização em vez do ranking global por
play_count acumulado. Se houver menos de 7 músicas com reprodução na última semana, mostra apenas as
disponíveis (secção pode ficar mais curta).

39. Algoritmo: Novos Lançamentos (últimos 10 dos artistas preferidos)

A secção 'Novos lançamentos' do Home apresenta os últimos 10 lançamentos (álbuns) dos artistas preferidos do
utilizador. 'Artistas preferidos' = os artistas da aba Artistas da Biblioteca, derivados do ListeningHistory do último
ano (loadRecentArtists, ver secção 30). Implementação: obter os favoriteArtistIds (artistas ouvidos nos últimos
12 meses), depois Album.filter({ artist_id: { \$in: favoriteArtistIds } }, '-release_year', 50) e cortar nos 10 mais
recentes por release_year desc. Isto personaliza a secção: só aparecem lançamentos de artistas que o utilizador
já ouve, em vez dos lançamentos globais mais recentes do catálogo. Se o utilizador não tiver artistas preferidos
(sem histórico), fallback para Album.list('-release_year', 10) globais para não deixar a secção vazia.